Project Description and ADRs

Course: 2110521 Software Architecture (2026/1)
Project: CampusKit

1. Project Name and Members

CampusKit is a peer-to-peer rental website for current Chulalongkorn University students. Students can list items they own, rent available items from other students, and arrange pickup and return on campus.

Name	**Student ID**
Chayutphong Soisri	6630054621
Thara Waranuset	6630132921
Thanyaporn Rattanacharoentham	6630138721
Pon Kittinaraporn	6630237121
Chalisa Wirojratana	6632045321

2. Project Description

2.1 Problem and Proposed Solution

Students may need equipment for only a short period, such as a camera for a course project, formal clothing for an interview, or a tent for a trip. Buying an item for occasional use can be costly, while other students may own suitable items that are currently unused.

Arranging rentals through social media and group chats can create several difficulties:

Problem	**How CampusKit addresses it**
Listings are scattered and hard to find across conversations.	A searchable catalogue brings rental offers together.
Owners must coordinate availability manually.	Date-based reservations prevent overlapping bookings.
Agreements and transaction status are unclear.	Both parties can view agreed terms, booking status, and rental history.
The item condition at pickup and return can be difficult to verify.	Pickup and return records preserve photographs and accessories; dispute adjudication is deferred.
Students have limited information about other participants.	Verified accounts and reviews provide a record of previous rentals.

These are the problems the project is designed to address, rather than findings from a completed user survey.

3. Target Customer and Actors

Actor	**Main responsibility**
Owner	List personally owned items, set availability, approve requests, and record handovers.
Renter	Find an item, request a rental, arrange payment, collect the item, and return it.
Platform Administrator	Review student verification and listing approval, and investigate failed payment or notification operations.
Payment Gateway	Authorize, capture, release, refund, and pay out funds through the payment integration.
Email Provider	Deliver verification messages, booking updates, and rental reminders.

Owner and Renter are roles of the same Student account. A student may perform both roles across different transactions but cannot rent their own item. Publishing and booking require verified current student status. An email verification code proves control of an address; student status is reviewed separately.
Scope for this iteration: implement listing, booking, payment, handover/return, reviews, and notifications. Report submission/management, damage/loss claims, dispute adjudication, and a claim-window workflow are deferred. Administrator verification, listing approval, and technical recovery remain in scope.

4. Use Cases

UC-1: Publish an Equipment Listing

Field	**Description**
Primary actor	Owner
Supporting actors	Platform Administrator, Email Provider
Goal	Make an item discoverable and bookable.
Trigger	The owner chooses to list an item.
Preconditions	The owner is signed in, verified, not suspended, and has a registered payout destination.
Success outcome	A published listing contains item details, prices, pickup options, and availability.

Main flow

1. The owner enters the item's name, category, description, condition, accessories, and replacement value.
2. The owner uploads photographs and sets the daily rate, deposit, and permitted rental duration.
3. The owner chooses campus pickup options and marks unavailable dates.
4. The system validates the information and saves both the listing and its availability.
5. The system screens the item. Flagged or high-value listings require administrator approval.
6. Once approved, the listing becomes searchable, and the owner receives a notification.

Alternative flows

- Incomplete or invalid information: explain the error and save a draft.
- Rejected listing: keep it unpublished and provide the reason. The owner may revise it for review.
- Availability cannot be saved: do not publish until calendar setup succeeds.
- An existing listing is edited or unpublished: stop new requests where appropriate, while preserving the terms of confirmed bookings. Cancelling those bookings follows the cancellation policy.

UC-2: Book and Pay for a Rental

Field	**Description**
Primary actor	Renter
Supporting actors	Owner, Payment Gateway, Email Provider
Goal	Reserve an available item for an agreed period.
Trigger	The renter needs an item for selected dates.
Preconditions	The renter is signed in, verified, eligible to book, and has a supported payment method. The listing is published.
Success outcome	A confirmed booking has reserved dates, agreed terms, successful authorization, and a rental record awaiting pickup.

Main flow

1. The renter searches by keyword or category and filters by rental dates and campus location.
2. The system displays matching available listings, prices, and owner review summaries.
3. The renter selects pickup and return times. The system displays the rental fee, deposit, cancellation policy, and late-fee rule.
4. On submission, the system rechecks eligibility and availability and atomically reserves the requested period.
5. The owner approves or declines the request within the response window.
6. On approval, the system retains the reservation and requests separate authorizations for the fee and deposit.
7. After authorization succeeds, the system creates the rental record, confirms the booking, and notifies both parties.

Alternative flows

- Dates are no longer available: reject the request without authorizing payment.
- Owner declines or does not respond in time: close the request and release the dates.
- Authorization fails: allow one retry within 30 minutes, ending no later than pickup. If unsuccessful, release the dates and reverse any successful partial authorization.

- Confirmation fails after authorization: retry safely or cancel and reverse the authorization. Track unresolved reversals for administrator attention.
- Cancellation before pickup: update the rental if one exists, and release the dates.

UC-3: Complete the Handover, Return, and Settlement

Field	**Description**
Primary actors	Owner and Renter
Supporting actors	Payment Gateway, Email Provider, Platform Administrator for exceptions
Goal	Record pickup and return, then complete settlement.
Trigger	A confirmed rental reaches its pickup time.
Preconditions	A confirmed booking exists. Payment readiness is checked before pickup and again at handover.
Success outcome	An accepted return is recorded, settlement succeeds, and the rental closes.

Main flow

1. The system checks payment readiness in advance and alerts the parties if action is needed.
2. At the agreed location, both parties record pickup photographs, condition, and included accessories.
3. The renter confirms the owner's one-time pickup code. After rechecking authorization and successfully capturing the fee, the system records pickup. The deposit remains reserved.
4. The system tracks the due time and sends a return reminder.
5. At return, the owner confirms the renter's return code. Both parties record the item condition and accessories, and the owner confirms an accepted return.
6. After an accepted return, the system records the returned item as available. Items not yet returned remain blocked from new requests.
7. The system settles any late fee defined in the agreed booking terms, releases the remaining deposit, and pays the owner. There is no claim window or automated damage/loss deduction in this iteration.

8. After settlement succeeds, the rental closes and both parties may submit reviews.

Alternative flows

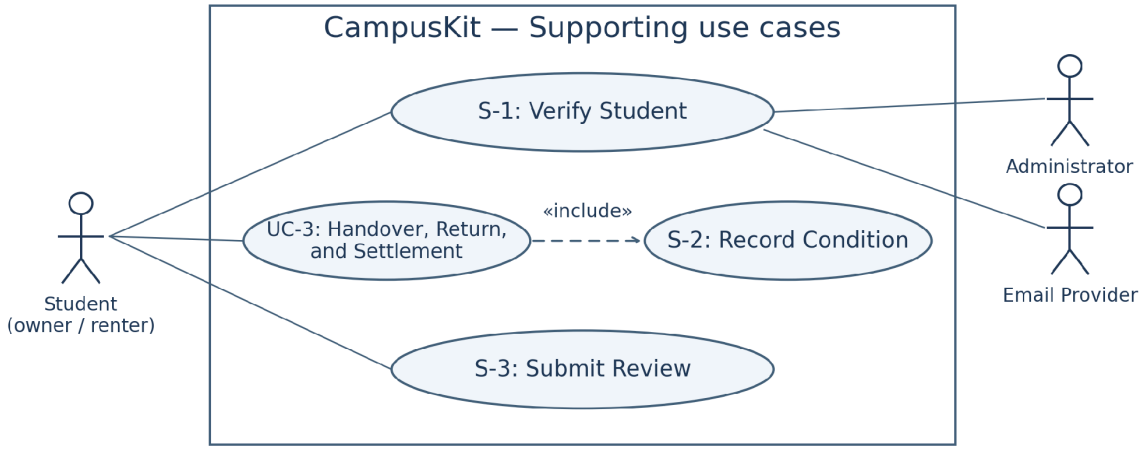
- Failed pickup: an invalid code or failed capture does not confirm handover. If the item is not as described, the renter may refuse it; cancel without a renter penalty and reverse payment.
- Late return: block new requests and identify affected future bookings. Notify those renters and offer an agreed reschedule or full cancellation without penalty. Keep the rental open until return is confirmed.
- Unaccepted return or missing item: keep the rental open and the item unavailable. Damage/loss claims and adjudication are outside the platform's current scope; the system does not determine compensation or deduct it automatically.
- Payment risk or settlement failure: if the deposit expires after pickup, preserve the custody record and alert the parties and administrator. Track partial settlement and retry only unfinished operations. Do not close the rental or report unsecured funds as collected.

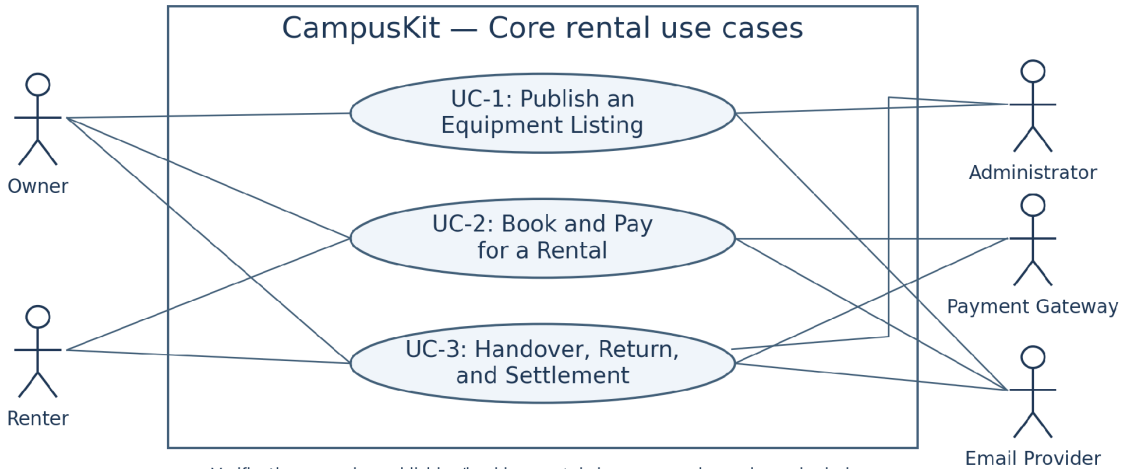
4.1 Supporting Use Cases

ID	**Use case**	**Actors and behaviour**
S-1	Verify Student	Student verifies their email; administrator reviews current student status. This precedes publishing and booking.
S-2	Record Condition	Owner and renter record pickup and return evidence. Included in UC-3.
S-3	Submit Review	Either participant reviews the other after rental closure. Closure is a precondition, not an extension relationship.

4.2 Use Case Diagram

The two diagrams show the core rental flow and supporting use cases within the current CampusKit scope. Actors are outside the system boundary; ellipses are use cases. Solid lines indicate participation, including administrator support for technical exceptions in UC-3. The supporting diagram repeats UC-3 to show its «include» relationship with S-2 Record Condition. Student verification precedes publishing and booking; rental closure precedes review submission.





Verification precedes publishing/booking; rental closure precedes review submission.

5. Key Requirements

5.1 Functional Requirements

ID	Requirement	Use cases
FR-01	Verify students and enforce account, participant, and administrator permissions.	S-1
FR-02	Manage listings, photographs, prices, moderation, and availability.	UC-1
FR-03	Search by category, keyword, dates, and campus location; show review summaries.	UC-2
FR-04	Generate and preserve quotes and atomically reserve dates; support approval, rejection, expiry, and cancellation.	UC-2
FR-05	Track fee and deposit authorization, capture, release, refund, and payout through a payment adapter.	UC-2, UC-3
FR-06	Record pickup and return through one-time codes and condition evidence.	UC-3, S-2
FR-07	Detect overdue rentals, calculate late fees, block unavailable items, and handle affected bookings.	UC-3
FR-08	Support participant reviews, booking/rental history, and itemised payment status.	UC-2, UC-3, S-3

ID	Requirement	Use cases
FR-09	Send updates and reminders; track failed payments and notifications for recovery and administrator review.	UC-1, UC-2, UC-3

5.2 Non-Functional Requirements

These are acceptance targets, not measured results. Performance testing uses 300 concurrent virtual users, 1,500 listings, a documented request mix, and the payment test adapter.

ID	Requirement and verification
NFR-01	Under the stated test load, p95 search latency is at most 1 second and single-resource reads at most 500 ms. Server errors remain below 0.5%; expected business rejections are counted separately. Verify by load testing.
NFR-02	Concurrent requests cannot create overlapping reservations. Retried operations cannot duplicate a capture, deduction, refund, or payout. Verify with concurrency and timeout-after-success tests.
NFR-03	External access uses HTTPS. Private evidence and transactions are restricted to authorised users; raw card data is not stored or logged. Verify with access-control tests and log inspection.
NFR-04	Each service owns its database and supports independent deployment through compatible API contracts. Verify database permissions and consumer contract tests.

ID	Requirement and verification
NFR-05	A notification outage does not reverse a successful rental operation. Failed deliveries and payment steps remain recoverable and traceable with correlation IDs. Verify by simulated outages and recovery.
NFR-06	The three core use cases work on a 360-pixel-wide viewport without horizontal scrolling, with labelled forms and keyboard access. At least four of five users with preverified test accounts can find an item and submit a request within 10 minutes. Verify by walkthrough and usability test.
NFR-07	Database recovery targets are an RPO of 24 hours and an RTO of 4 hours. Reconcile unresolved payment outcomes after recovery before retrying money operations. Verify through a restoration exercise.

The website supports Thai and English and uses THB. Before operation with real users, the team must validate identity verification, payment-provider capabilities, privacy notices, and data retention rules. These are deployment dependencies rather than claims of completed integration.

6. Architecture Decision Records

The seven records below follow the Nygard structure: Context, Decision, Status, and Consequences. Each record states the alternatives considered, the selected approach, and its trade-offs. All seven decisions are Accepted. Payment-provider selection and the production deposit strategy remain separate, deferred decisions.

ADR-001: Decompose by Business Capability

Context: CampusKit has a five-person team and one term to deliver independent services. A modular monolith offers simpler deployment and local transactions; microservices make independent deployment and failure recovery explicit but require more contracts and operations. Use service decomposition as the course architecture, with business responsibilities listed here as boundaries rather than as separate ADRs.

Decision: Use seven independently deployable services: Student Identity, Listing, Booking, Rental, Payment, Review and Reputation, and Notification, behind one API gateway. Student Identity owns verification and account standing. Each service exposes APIs and owns its database; no service reads another service's tables. Report handling and dispute adjudication are deferred and have no service in this iteration.

Status: Accepted.

Consequences: Separate deployment and data ownership support the course's architecture goals. The team must assign owners to seven services, maintain API contracts and a common local startup configuration, and test cross-service failures with correlated logs. This accepts more operational work than a modular monolith; reducing the feature scope is necessary to keep the course implementation manageable.

ADR-002: Authenticate Centrally, Authorise at the Data Owner

Context: Gateway-only authorisation centralises checks but cannot reliably evaluate current resource ownership without duplicating service data. Checking every permission at each service keeps decisions next to the protected resource but adds repeated checks and internal identity configuration. A valid login alone does not establish listing ownership or rental participation.

Decision: Authenticate at the gateway and propagate a verifiable identity context over authenticated internal connections. Services reject missing or unverifiable context and ignore client-supplied identity headers. At each protected endpoint, check the caller's role and current resource ownership or rental participation before reading or changing data. Deny access by default; internal calls do not bypass these checks.

Status: Accepted.

Consequences: Resource permissions remain with the owning service. The team must configure trusted identity propagation, secure internal routes, and test forged identities, unauthorised direct calls, and access to another student's records. Every new resource requires an explicit authorisation check.

ADR-003: Prevent Overlapping Reservations with Database Constraints

Context: Owner blackouts, pending requests, and confirmed bookings compete for one item. An application-level check followed by an insert can race under concurrent requests. Serialising requests with a distributed lock adds lease and recovery complexity. A PostgreSQL exclusion constraint enforces non-overlap at the storage boundary but requires range modelling and conflict handling.

Decision: Keep the authoritative calendar in Booking. Represent blocking periods as half-open intervals [start, end), and enforce non-overlap for the same item with a PostgreSQL GiST exclusion constraint on item identity and the time range (using btree_gist for scalar identity). Store owner blackouts and active reservations in the constrained calendar. Create or change a reservation within one transaction; convert a constraint conflict into an unavailable-dates response. Expiry or cancellation explicitly removes the blocking interval in a transaction. Listing publication waits for calendar setup, and search availability remains advisory.

Status: Accepted.

Consequences: The database prevents overlapping committed periods even when requests arrive together. Back-to-back rentals are permitted by half-open intervals. The team must test simultaneous overlapping inserts, adjacent intervals, owner blackout changes, and expiry-versus-confirmation races. An expiry worker releases stale reservations; a delayed worker can temporarily reduce availability but must not allow double booking. Booking remains a dependency for publication and reservation.

ADR-004: Database per Service

Context: Shared tables simplify joins but couple service schemas; separate databases support clear ownership at the cost of cross-service coordination. PostgreSQL and MySQL are relational candidates for structured bookings and payments. A document database is another option, but these workflows do not currently need a document-first model. The proposed baseline is PostgreSQL because the design prioritises relational transactions and a consistent database setup across services, not because MySQL is unsuitable.

Decision: Use PostgreSQL with a separate logical database and credentials per service. A single PostgreSQL instance may host these databases in the course environment. Cross-service access uses published APIs.

Status: Accepted.

Consequences: Each service controls its schema. The team must create separate credentials, maintain per-service migrations and backups, and test transaction boundaries. Cross-service reads require APIs and failed workflows require recovery logic. Hosting the logical databases on one server reduces setup work but leaves a shared infrastructure failure point.

ADR-005: Orchestrate Workflows with Idempotent Recovery

Context: Booking confirmation and settlement cross service boundaries. A timeout does not prove failure, and a local rollback cannot undo a completed remote payment. Event choreography would distribute the sequence across services; a dedicated orchestrator would add another deployment. Keeping orchestration with the service responsible for the outcome makes the initial workflows easier to locate and test.

Decision: Booking orchestrates confirmation and cancellation; Rental orchestrates handover and settlement. Persist each workflow's current step, completed results, and stable operation identifiers before external calls. Payment enforces a unique identifier per financial operation and returns its recorded result on a completed duplicate request. After a timeout, mark the outcome uncertain and reconcile through the adapter before retrying. Resume unfinished steps; compensate for a failed confirmation by reversing successful authorisations and releasing the reservation. Settlement begins after an accepted return and has no dispute-service dependency.

Status: Accepted.

Consequences: The team must store workflow progress and operation identifiers, implement compensating actions, and test timeouts after remote success. Partial settlement resumes from unfinished steps. Recovery failures need an administrator queue and a documented resolution procedure. Booking and Rental gain additional workflow logic, but completed money operations are not repeated blindly.

ADR-006: Isolate Payment Integration Behind an Adapter

Context: Marketplace payouts and authorization durations depend on the provider. Direct provider calls throughout the application would couple business rules to one SDK. An adapter requires extra mapping code but permits sandbox and production implementations behind the same contract. Selecting an unverified renewal interval would make deposit protection uncertain.

Decision: Payment owns the adapter, transaction ledger, and reconciliation. Demonstrate separate fee and deposit authorizations, fee capture at pickup, and settlement after an accepted return using sandbox/test operations. Model authorization expiry explicitly rather than assuming a hold lasts throughout every rental. Provider selection and the production deposit strategy remain deferred.

Status: Accepted.

Consequences: The team must define the adapter contract, implement sandbox/test outcomes, and test expiry, declined authorization, partial settlement, and reconciliation. Production integration requires validating renewal and payout support. Failed coverage after pickup needs a payment-risk workflow that preserves custody and distinguishes secured funds from shortfalls.

ADR-007: Frontend Framework

Context: The project needs responsive student and administrator web interfaces for listings, booking, handover, and payment status. Compare React with TypeScript and Vue 3 with TypeScript on component structure, setup effort, and learning cost. Both support TypeScript; using one language across the stack does not distinguish them.

Dimension	React with TypeScript	Vue 3 with TypeScript
Component structure	JSX/TSX keeps rendering logic close to TypeScript; reusable typed components and hooks.	Single-file components group templates, logic, and styles; reusable logic through the Composition API.
Advantages for this project	Retains the proposed React baseline and supports shared listing cards, booking forms, and status views.	Provides an explicit template structure and component organisation suited to the same interfaces.

Dimension	React with TypeScript	Vue 3 with TypeScript
Costs and learning	Requires agreed routing/data-fetching tools, plus JSX, hooks, and state conventions.	Requires learning template directives, reactivity, and .vue files; switching would change the proposed frontend conventions.
TypeScript support	Supported; a shared language is useful but does not distinguish the two choices.	Supported; a shared language is useful but does not distinguish the two choices.

Selection rationale: retain the proposed React baseline to avoid changing direction within one term, and accept the extra tooling decisions. Vue remains a viable alternative. This is a project-specific continuity trade-off; it does not assume greater React experience or a universal performance advantage.

Decision: Use React with TypeScript for the student and administrator web interfaces. Keep one shared frontend project with reusable components and a documented routing, API-access, and form-handling convention. Backend framework selection is outside this frontend ADR.

Status: Accepted.

Consequences: Reusing components and typed API models can keep the web interfaces consistent. The team must document dependency versions, build/test commands, routing and data-fetching choices, and loading/error states. Members unfamiliar with React need onboarding time, and JSX/hooks require agreed review conventions. Validate NFR-06 with keyboard access, labelled forms, and a 360-pixel viewport. The frontend consumes published APIs and does not contain authoritative permission, availability, or settlement rules.